ML Final Project: Predicting How Integrated Alumni from Masters Programs are into the Job Market

Celina Madaschi, Victoria Gutierrez, Catarina Gaspar

April 3, 2026

1 Introduction

In this report we aim to predict how successful students will be after the completion of a masters in program. To answer this question we consider the French Ministry of Higher Education's data set on Professional Integration of Master's graduates in universities and similar institutions. We have open access to this data through data.gouv.

The information compiled on this data set was obtained through a nation-wide Ministry of Higher Education survey conducted 18 months or 30 months after the program's completion (this is a specified variable in the data set.) The survey asked students from each program whether they were employed, their salary, and whether they completed their Masters under a scholarship. This information is aggregated such that the data set considers the employment rate, the employment rate particular to paid French vacancies, the median net salary, and the percentage of students who are employed in upper class or middle class jobs for each program. Note that because some Masters programs have very few students which participate in the study, the Ministry of Higher Education only assigns such estimates for programs which had over 30 respondents in the survey and whose response rate was above 50%. Information on the student's career status is then used to estimate the insertion rate of alumni into the job market. The Ministry of Higher Education defines their calculation of the insertion rate of a program by the percentage of alumni currently in the job market relative to all people in the job market with a diploma.

The data set also contains specific administrative information on the program, such as the discipline and department the program falls under, the university the program is conducted by, the location of the program, the type of diploma obtained through this masters, and the percentage of women in the program. Additional information, such as the median salary and employment rate respective to the university's location is obtained through INSEE statistics. The full table of variable names and descriptions can be found in the appendix.

As our data set is restricted to people who graduated with a Masters from a French university (continental and extra-territorial France) between 2010 and 2020 our prediction results are most valid for students in French Masters programs who participated in a program in the given period or slightly before or after.

2 Exploratory Data Analysis

Before discussing the exploratory data analysis conducted prior to the construction of prediction algorithms we briefly discuss the data cleaning performed.

We first removed all observations which had less than 30 responses or a response rate lower than 50%, as the Ministry of Education did not assign integration rates or employment rates to such statistics. This significantly decreased the number of observations we consider from 19,603 to 10,584.

After this we removed columns from our data set which had more than 30% missing values over all observations. The columns this compromised were `remarque`, `etablissementactuel`, `taux_d_emploi`, and `taux_d_emploi_salarie`, where the high number of missing values for the two latter variables comes from them only being recorded in recent years. We also removed variables `salaire_net_median_des_emplois_a_temps_plein` and `salaire_net_mensuel_regional_3eme` from our data set as these two variables were extremely correlated with `salaire_brut_annuel_estime` and `salaire_net_mensuel_regional_median` respectively. Specifically, they each had correlations of over 0.99. These results, along with correlations between the remaining variables, are presented in Figure 12 located in the appendix.

2.1 What does the distribution of insertion rates look like?

Before defining the target variable, it is important to understand how insertion rates are distributed across the dataset. Figure 1 shows this distribution for all program cells and time points.

The distribution is strongly left-skewed, with most programs reporting rates above 80% and a mode around 92–94%. Very few programs fall below 85%. This means that truly low-integration programs are a **minority class** (roughly 25% of all observations) which has direct implications for how models must be evaluated. Accuracy alone will be misleading; metrics such as F1-score, ROC-AUC, and confusion matrices will be necessary to properly assess performance. The 25th percentile (P25 = 86%) is therefore used as the binary threshold to define the positive class.

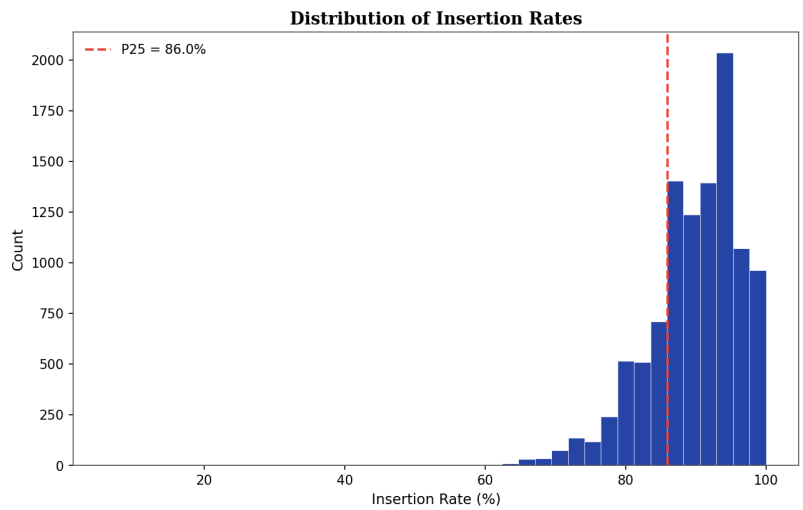

Figure 1: Distribution of insertion rates (%). The red dashed line marks P25 = 86%, the threshold used to define *low integration*.

2.2 Does the class balance change depending on the survey timing?

The dataset contains observations at both 18 and 30 months after graduation. Figure 2 examines whether the shape of the two classes shifts between these two waves.

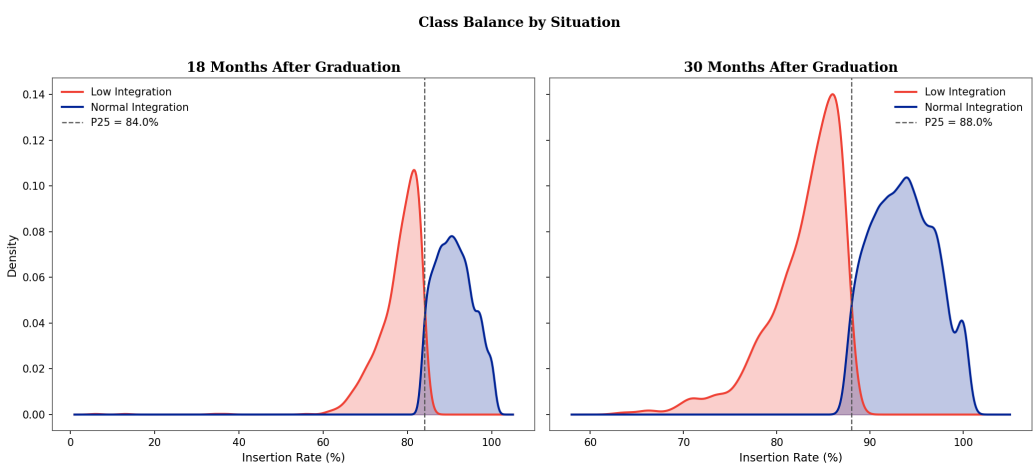

Figure 2: Kernel density estimates for low-integration (red) and normal-integration (blue) programs at each survey wave. Dashed lines mark the P25 threshold per wave.

From the two graphs we can see that threshold shifts upward over time: P25 = 84% at 18 months versus P25 = 88% at 30 months, reflecting the fact that graduates continue finding employment after the first survey,

suggesting that the 18-month insertion rate can be used as a **predictive feature** for the 30-month outcome, rather than being treated as an alternative version of the target.

2.3 How do insertion rates differ across fields of study?

Figure 3 and Figure 4 jointly address the role of the academic field, both in terms of cross-sectional distribution and temporal evolution.

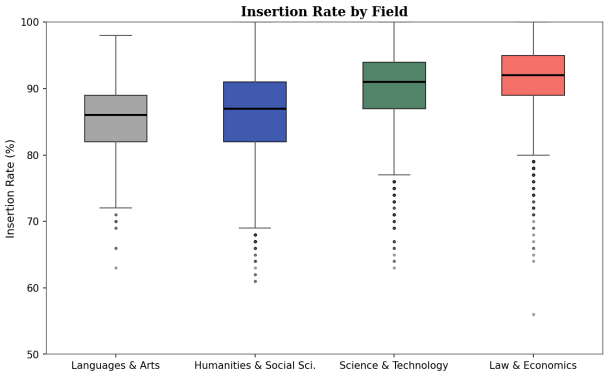

Figure 3: Insertion rates (%) by field of study.

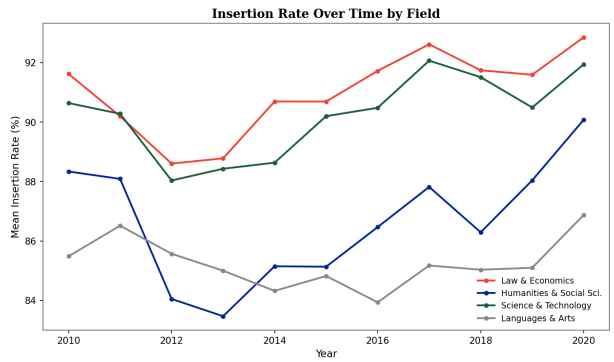

Figure 4: Mean insertion rate over time (2010–2020) by field.

Law & Economics and **Science & Technology** consistently show the highest median insertion rates (≈92% and 91% respectively) and tight interquartile ranges, which indicates homogeneous outcomes. **Languages & Arts** presents the lowest median (≈86%) and the widest spread, pointing to high heterogeneity within the field. **Humanities & Social Sciences** sits in between, but with a long lower whisker and several downward outliers, revealing a subset of programs with persistently poor outcomes.

Over time, all fields share a common dip around 2012–2013, likely reflecting the macroeconomic aftermath of the 2008 financial crisis. Humanities & Social Sciences is the most volatile, while Law & Economics shows a steady upward trend reaching above 93% by 2020. These patterns confirm that **field of study and graduation year** are both strong predictors that should be included in every model.

2.4 Are salary differences consistent with insertion rate differences?

Figure 5 shows the distribution of median net monthly salaries by field, which allows us to check whether the field ordering observed in insertion rates is also reflected in earnings.

The salary ordering mirrors the insertion-rate ordering almost exactly: Law & Economics and Science & Technology command median salaries of around 2,000/month, while Languages & Arts and Humanities & Social Sciences sit near 1,650–1,700/month. This consistency suggests that the field variable captures both the *quantity* of employment (whether graduates find work) and its *quality* (what kind of work they find). High-salary outliers above 3,000/month are present in all fields but are more common in Law & Economics and Science & Technology, consistent with the greater variance in their upper tails.

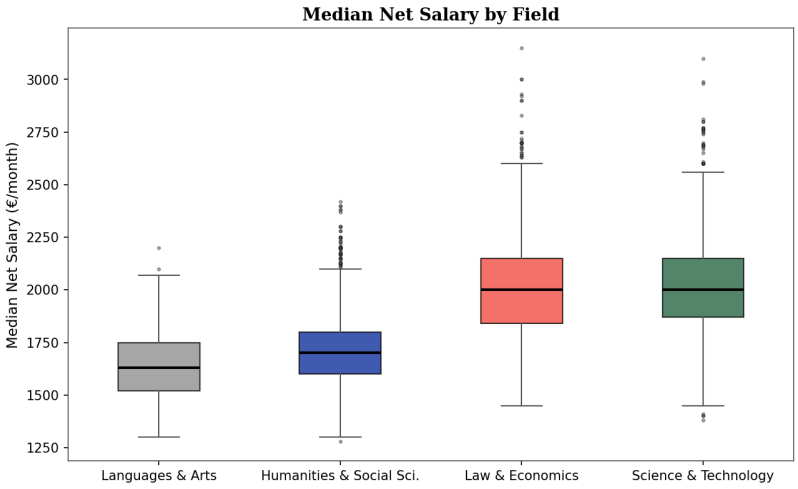

Figure 5: Median net monthly salary (/month, full-time) by field of study.

2.5 Do employment quality indicators predict insertion rates?

Figure 6 examines the relationship between insertion rate and three employment quality indicators: share of upper/intermediate-level jobs, share of stable contracts, and share of full-time employment.

Figure 6: Insertion rate (%) against three employment quality indicators by field. Lines show OLS fits per field. The dashed line marks the low-integration threshold (P25 = 86%).

All three panels reveal a **positive association**: programs where more graduates hold stable, full-time, or skilled positions also tend to achieve higher insertion rates. However, the slope varies across fields. Science & Technology shows the steepest gradient for upper/intermediate jobs, while Languages & Arts has a flatter relationship with full-time employment. Notably, observations below the P25 threshold are present across all fields, confirming that low integration is not exclusive to any single discipline.

It is a good sign that the three features are correlated with the globalizing insertion rate of a program as it suggests that these variables are accounted for in the insertion rate calculation. For what follows we use a program's insertion rate as our target variable.

3 Predicting Masters Programs' Integration Rate as Continuous Variable: Using Lasso and Ridge Regressions

After concluding that using the program's insertion rate is an appropriate variable for measuring the program's success, we begin our prediction analysis.

3.1 Linear Regression:

As a first step, we predict the program's integration rate as a continuous variable. We predict this to be challenging as even though a program's integration rate can vary between 0 and 100, in our data set the interquartile range for this variable is only of 9 percentage points, and the respective standard deviation of about 6.7 percentage points. We expect this low variation to potentially be problematic as our data doesn't capture many programs with low insertion rates, making it harder to predict and identify what causes a program to be as such.

We start our prediction of a program's integration rate with the most interpretable continuous ML model: that is a simple Linear Regression as we believe this will be a good base to build from.

Although a linear regression model can be classified as a method of frequentist statisticians, it also applies to Machine Learning. This is because we consider the set of possible linear functions as the set of possible prediction functions. Then, for a set of training data X_{train} and Y_{train} the algorithm chooses the prediction function $\hat{f}(X) = \beta_0 + \beta^t X$ such that the empirical risk,

$$\frac{1}{n}\sum_{i=1}^{n} l(\hat{f}(X_{train_i}), Y_{train_i}),$$

is minimized.

In our case, we choose the loss function, l, which we work with to be the mean squared error; meaning that even though we are looking at this linear regression through machine learning lenses, the predictor function which we optimize is equivalent to the $\hat{\beta}$ obtained through OLS.

As discussed in Section 2, we exclude identifier variables, variables which also correspond to response features, and variables which are highly correlated with each other from our data set, such that we regress a cohort's integration rate solely on the year of the program, the type of diploma the program offers, the university of the program, the location of the program, the department the program falls under, the discipline of the program, the number of months that has passed since the cohort graduated, the percentage of students with a scholarship in the program, the percentage of women in the program, the unemployment rate of the university's region, and the median and first quantile monthly salary in the university's region. Although `explois_exterieurs_a_la_region_de_luniversite` is not a proxy for determining how well off the alumni from a program are, such as the median salary of people who studied in the program or the employment rate of people who studied under the program, it is still an outcome from the program. We therefore exclude this variable in our analysis as we don't want an outcome of the program to predict how well integrated students of the program would be, as students would not have access to this information prior to entering the program. Finally, we also regress the program's outcome on a proxy for the program size. We proxy the program size by dividing the number of respondents to the survey for a given program by the response rate. Finally, we use a sampling weight variable, provided in the data set by `poids_de_la_discipline`, to weight observations.

Overall, because we want to estimate both the bias and variance of modelling our predictions through a linear regressor, we split our training data into five folds, such that we can fit the model with 80% of the training data, and test it with the remaining 20% of the training data. Because we split the data into 5 folds, we do this five times.

Over the 5 folds, the mean score of the predictor, measured through the MSE, is of 25.38, while the variance of this score over the 5 folds is of 3.7. This means that our linear regression model is an improvement over predicting the program integration rate by random selection as the variance of integration rates for a sample the same size as that used in cross validation (i.e. 20% of the training data) is of about 46.33. However, we aim to improve on these results, and thus consider whether considering a linear regression model with ridge or lasso penalization terms improves our results.

3.2 Ridge Regression:

Initially, it may seem strange to consider that adding a Ridge penalization term to our regression would improve the results of our model. This is because a Ridge Regression traditionally aims to decrease the noise within the variables which we use to explain our predictor by considering a β which minimizes the MSE penalized by the size of β multiplied by a parameter λ. That is, in Ridge Regression, our algorithm aims to find a predictor defined by β which minimizes

$$\frac{1}{n}\sum_{i=1}^{n}(Y_{\text{train}_i} - (\beta_0 + \beta^t X_{\text{train}_i}))^2 + \lambda\left(\sum_{j=0}^{p}\beta_j^2\right), \tag{1}$$

where p corresponds to the number of parameters we must estimate, and λ is a parameter we must tune. However, because in the forefront it appears that we are regressing the integration rate only on 13 variables, it may seem unnecessary to do this. Nevertheless, because six of these variables are actually non-ordinal categorical variables, our regression actually estimates 138 parameters as opposed to 13. Thus, it seems that a Ridge Regressor could in fact be impactful in our regression by removing some noise.

We therefore run a Ridge Regressor with 7 different possible values for λ; namely: 0.01, 0.1, 1, 10, 100, and 1000, 10000. The closer λ is to 0, the closer our predictor is to that of the simple linear regression, whereas the larger λ is, the closer predictors shrink to 0, with only a few remaining significant. In order to fine tune our Ridge regressor to the best λ value, we run the same 5-fold analysis used for the linear regression, with the caveat that this time we run such a cross validation not only to consider the variance in the efficacy of our predictor, but also in order to tune λ.

Upon running this cross validation procedure we find that our ridge regression model is expected to perform best for $\lambda = 100$, with a mean score of 25.37, and variance of 3.9. Therefore, it seems that if our Ridge regression improves the performance of our Linear Regression model, it does so likely insignificantly.

Out of curiosity, we consider which parameters are the least shrunk by the Ridge regressor as λ increases. We decide to plot this as we hypothesize that this will give us insight in which predictors are most important in predicting the integration rate of a program. Such considerations can be made using Figures 13 and 14 in the Appendix.

Figures 13 and 14 consider parameter estimates under ridge regression as the value of λ increases. In 13 we consider λ values between 1^{-10} and 300, as our optimal λ value is estimated to be of 100, while in 14 we consider λ values between 10^{-10} and 10^{14}. We can see from 13 that estimates barely change for λ values between 10^{-10} and 300. This aligns with the lack of improvement of a MSE under a Ridge Regression with $\lambda = 100$.

From Figure 13 we also see that at the stage where $\lambda \approx 100$, the diploma type of the masters program; establishments: *Toulouse 1 - Capitole*, *Lille 2 - Droit et Sante*, *Paris Pierre et Marie Curie*, *INU Jean-Francos Champollion*, *Avignon*, and *Evry Val d'Essone* with reference *Nice - Sophia-Antipolis*; and management, informatics, history and geography, teaching, psychology and earth disciplines with other financial law disciplines as reference are deemed to have the highest absolute value impact on the integration rate of a program per unit change. Note; however, that these conclusions can be a bit misleading, as a unit change in the median salary of a region or unemployment rate of a region is much smaller, in comparison to how much these units can change, than a one 'unit' change in the program's department. Therefore, while the corresponding $\hat{\beta}$ estimate for the unemployment rate in the program's region might be smaller than the estimate which distinguishes a history and geography masters from that of a financial law masters, this does not mean the unemployment rate in the program's region has a smaller impact on the program's integration rate as our units of comparison for the explanatory variable are different. Whilst this is taken into account through `Standard Scalar` in our code, such that λ is not more penalizing of features with lower variance, the $\hat{\beta}$ estimates for each parameter in Figure 13 are still true to the true estimates. Thus, in order to analyze which features have the greatest impact, we instead consider the features which are shrunk the least in the long run. For this, we turn to Figure 14.

In Figure 14 we have instead highlighted variables corresponding to the estimates whose value at $\lambda = 10^{14}$ is greatest compared to its value at $\lambda = 10^{-10}$. This allows us to account for differences in estimate sizes due to differing units of the explanatory variables used. Thus, from 14 we gather that the region's employment rate and salary distribution, as well as the program's size are key variables in predicting the program's integration rate. Specifically, we estimate that the insertion rate of programs located in regions with low unemployment and higher salaries is higher than those located in regions with high unemployment and low salaries. Further, we predict that Master's programs of diploma type LMD have lower insertion rates than those of diploma type ENS.

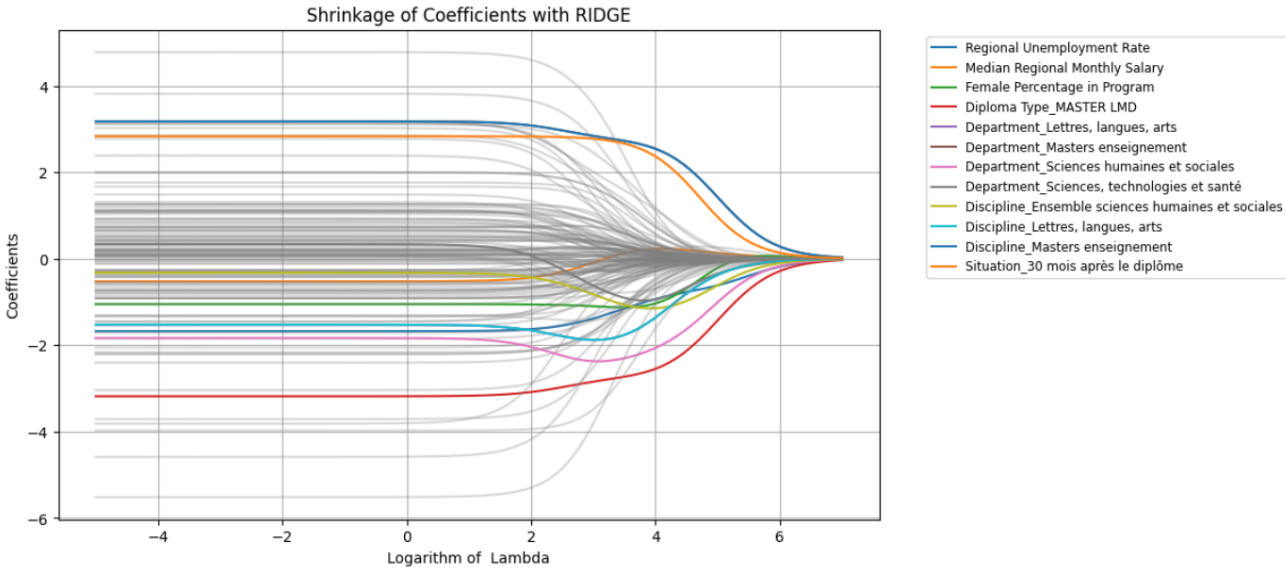

Figure 7: Estimates of parameters under ridge regressor as lambda increases. Estimates which correspond to those who size was greater than 0.005 in absolute value at $\lambda = 10^{14}$ are highlighted. Note that the logarithm considered is that of base 100.

Another approach which we could have taken as a work around the units problem is to consider which variables still have estimates higher than a base estimate lower than the lowest estimate for the lowest λ value. This is what we do in Figure 7. From this Figure, we also gather that the number of months passed since graduation (discretisized between 18 and 30), and the percentage of women in a Masters program are key in predicting the program's integrability. Specifically we expect that a cohort's insertion into the labor market is predicted to be higher the more months that have passed since graduation. We also expect that programs with a higher share of women are predicted to have lower insertion into the labor market. This possibly reflects a gap of entry into the labor market between genders.

Overall, we decide to present both Figures as they present slightly different insights, and hinder at the lower interpretation that comes with Machine Learning as it hard for us to pick which of the two methods clearly identifies which factors influence a program's integrability the most.

3.3 Lasso Regression:

Although a Ridge Regression did not improve our predictions of program integrability in a significant way, for completeness we measure whether a Lasso regression makes any improvements. Lasso regression is similar to Ridge regression in that it also adds a penalization term for noise. However, whilst a ridge regression does this by adding the squared norm of β scaled by λ to the loss function, Lasso instead does this by adding the norm of β scaled by λ to the loss function. That is, in a Lasso regression we try to find the predictor function associated with β which minimizes

$$\frac{1}{n}\sum_{i=1}^{n}(Y_{\text{train}_i} - (\beta_0 + \beta^t X_{\text{train}_i}))^2 + \lambda\left(\sum_{j=0}^{p}|\beta_j|\right). \tag{2}$$

In consequence, this makes a Lasso regression differ from a Ridge regression because for small β, the norm of β is generally greater than its squared norm. Thus in order to minimize (2), parameter estimates shrink to 0 faster than they would in (1). Therefore, while in a Ridge regression coefficients do shrink, they don't shrink to 0 as they would in a Lasso regression.

We thus run a Lasso regression for the same predictors and target variable as done for the Ridge Regression and simple Linear Regression. Similarly to the Ridge Regression specification, we split our training data into five folds such that we estimate what λ value corresponds to that with the highest score. Upon trying λ values of 0.0001, 0.001, 0.01, 0.1, 1, 10, 100, we find that our Lasso regression is expected to predict integration rates best

at $\lambda = 0.001$. The corresponding mean MSE over the 5 folds of our training data is of 25.37 while the variance of the MSE for $\lambda = 0.001$ over the 5 folds is of 3.90. Thus, the Lasso regression presents a similar performance to the Ridge and simple Linear Regressions. We note here that it makes sense that the optimal λ value for the Lasso regression is smaller than that of the Ridge as λ values shrink β faster in Lasso regressions than Ridge regressions.

3.4 Comparing the Three Models

Overall, the mean MSE over the three models only varied by about 0.02, which really is not substantial given that the variance of the target variable is of about 46.3 points. Thus, the bias of the model did not improve in a significant way when trying to control for noise through Lasso and Ridge regressions. Similarly, the variance of our original model did not improve under Ridge and Lasso regressions, rather it increased by about 0.2 points.

Although the mean MSE is slightly higher for the Linear Regression model, because the variances for the Lasso and Ridge Regression were raised by a higher amount than the mean MSE, we expect that this is not suggestive of a higher performing-model due to high variance we estimate, of about 3.8, between training and testing sets.

After fitting each model with the optimal λ on our training data, we test our predictions using the remaining testing data set. We find that mean-squared error between our predicted integration rates and the actual testing rates to be of 22.89 when we use a simple linear regression and Lasso regression alike, and to be of 22.92 for the Ridge model. Although the MSE was highest for the Ridge regression, it was only slightly so, pointing that there is little differentiability between the prediction power of the three models. On a positive note, we see that the MSE on our prediction data set was lower for the three models than the MSE calculated over cross-validations; therefore suggesting that our model is not overfit by the data.

Even though our linear regression models are an improvement from just predicting a program's integrability by random selection from the density distribution of program integrabilities, it still leaves a lot to be improved. We could have used partial least squares in order to remove some noise in our data set by identifying directions of movement within our explanatory data which are correlated with the integration rate of programs, such that instead of trying to eliminate specific variables from our regression, we would have tried to hone in on specific linear combinations of variables that affect our target. We could have also considered predictor functions which are not linear in nature; for example, we could have opened our class of prediction functions to polynomial functions as well. Although we do not do an in-depth analysis of these methods we note here that we did trial them and they out-performed the described models. Particularly, when find that tuning a ridge kernel regression to a cubic polynomial with $\lambda = 1$ and $\gamma = 0.1$, we find a mean MSE over 5 folds of only 21.03 with a corresponding variance of 3.42, suggesting this model is less biased and more adaptable than our linear model. Further, this suggests that the relationship between a program's insertion rate and the explanatory variables we have discussed is more complex than what linear relationship can capture. However, when identifying predicted insertion rates from these types of models we find a similar problem to that which can be identified in our simpler linear models. That is: program's insertion rates are predicted to be over 100%. It makes sense that this is an output of our models as we have not made such a restriction when constructing them, however such predicted values are unrealistic and present an important feature of our target variable. That is, though our target variable is continuous it is bounded by 0 and 100. This, in part motivates, an encoding of our target variable, where such boundaries can be appropriately dealt with.

4 Binary Outcome Modeling: Predicting Vulnerability in Professional Integration of French Master's Graduates

When reflecting on our data we consider the possibility that students or governments might not actually care about the numerical value of a Master's program's integration rate. Rather than predicting the exact insertion rate, the problem is framed as a binary classification task: a program is either vulnerable (low insertion rate) or not vulnerable (high insertion rate). We consider a program's integration rate to be low if it lies below 86%, which corresponds to the 25th percentile. Thus, it represents programs which truly are in the lower-tail of program integrability. We make this definition as we deem this to be a balanced cutoff. That is, whereas we see a cutoff of 90% to be too demanding, as some students may not desire to go straight into the job market after completing their masters, we see a cutoff of 70% to be too accommodating.

The binary target variable *taux_dinsertion* is constructed as:

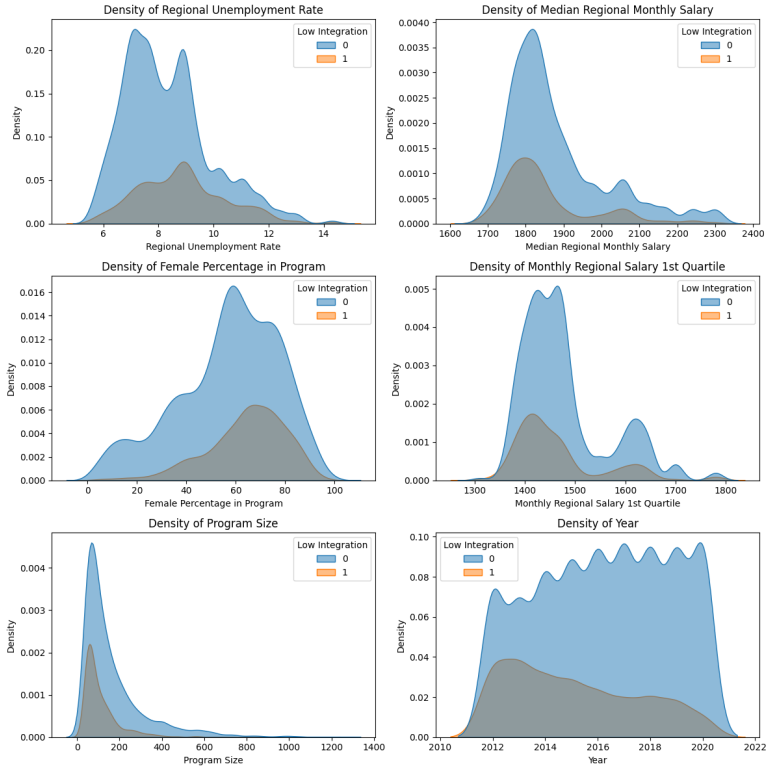

Figure 8: Distributions of regional unemployment rate, median and first quartile monthly salary, percentage of females in program, program size, and cohort year once conditioned on whether people in the program became well-integrated into the job-market at the time of the survey.

- **1:** Programs with an insertion rate at or below the 25th percentile (P25 = 86%). These are the programs where graduates have the greatest difficulty entering the labor market.
- **0:** All other programs.

The resulting class split is approximately 25.4% vulnerable and 74.6% not vulnerable.

4.1 Our data under a binary classification:

As a first step to understanding how this binary definition of program integration impacts our data, we consider how the numerical and categorical features are split between programs with a high and low integration rate. In Figure 8 we see that the distributions of low-integrated programs and well-integrated programs largely align over numerical features, with the distributions of regional unemployment rates and year of graduation varying the most.

In Figure 9, we see that the distributions of categorical features deviate once conditioned on program integration. For example, we see that there are no programs which are of low integration under an ENS Master diploma type or are considered a teaching masters. Further, we see that the majority of well-integrated cohorts counts were surveyed 30 months after graduation, whereas the majority of low-integrated cohorts counts were surveyed 18 months after graduation. More observations could be made but as to not extend our report, we limit our stated observations to those preceding. Overall, the conclusions which we take from Figures 8 and 9 are logical and align with our shrinkage coefficient analysis in Section 3.

4.2 Logistic Regression:

After better understanding our data once conditioned on the encoded program integration rates, we next try to produce a prediction function which predicts whether a program is well-integrated or low-integrated on the basis of the same features discussed in Section 3. Given that our target variable is now a binary variable, our

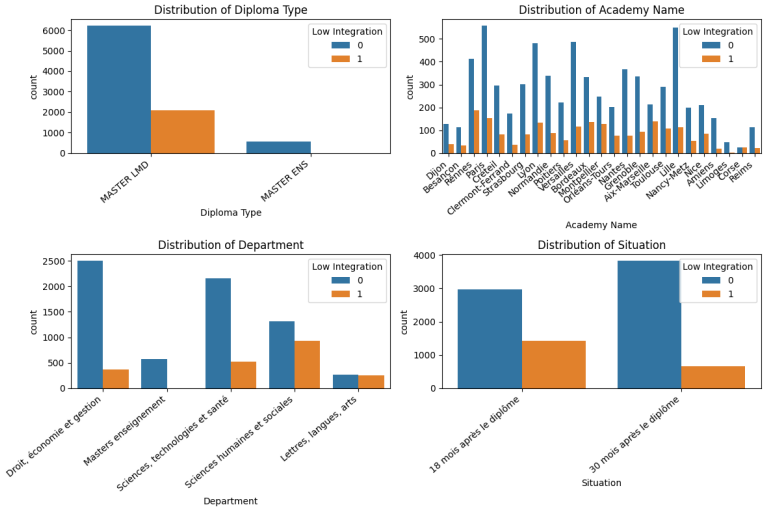

Figure 9: Distributions of diploma type, program location, department of program, and months since program completion once conditioned on how well integrated into the job-market the program's cohort was at the time of the survey.

prediction function is considered a classifier. Further, because our target variable can now only take values 0 or 1, instead of defining our loss function to be the mean squared error, we define it to be the 0-1 loss function. That is, we want our classifier $\hat{f}$ to minimize

$$\frac{1}{n}\sum_{i=1}^{n} \mathbb{1}\{\hat{f}(X_{\text{train}_i}) \neq Y_{\text{train}_i}\}, \tag{3}$$

In the case that all classes are equally balanced within our data set, this is in theory equivalent to classifying X_{train_i} to the most likely class conditioned on X_{train_i}. The challenge thus relies on identifying the probability distribution of our binary target variable once conditioned on our explanatory variables.

In a logistic regression learning model, where our target variable is binary, we assume that $P(Y|X = x)$ follows a logistic distribution. This implies that an estimate $\hat{\beta}_j$ corresponding explanatory variable X_j corresponds to the estimated impact of increasing X_j by one unit on the log-odds of Y.

Because in a binary classification method there are only two classes, then the most likely class is that whose probability is above 0.5.

However, when our data set is unbalanced and our data does not follow a logistic regression perfectly, the estimated $P(Y|X = x)$ are skewed such that predicting the majority class is more likely. This is because in essence our classifier aims to minimize (3), but when classes are imbalanced, predicting the majority class correctly minimizes (3) to a greater degree. Given that in our data set classes are imbalanced, as low-integrated programs are less represented in our data, we keep this in mind.

After discussing the theoretical underpinnings of our model, we are ready to apply it to our study.

As done for our Linear Regression model, we split our data between a training data set and a testing data set. The training data set corresponds to 90% of the original data set, whereas the testing data set corresponds to the remaining 10% of our data. As our target variable of low-integration is highly unbalanced in our original data set, we take the time to ensure that we don't have an unlucky random selection draw and verify whether the share of low-integrated programs in our training and testing data sets is similar to that of the original data set. This seems to be the case as only around 24% of programs in each of our training and testing data sets consist of low-integrated programs.

After splitting our data set into training and testing, and verifying that the distribution of program integration in each set is similar to our wider data distribution, we begin our analysis of logistic regression models on program integration prediction.

We start with the most simple form of a Logistic Regression model: one where we have not reweighted classes in our data such that the distribution of low-integrated and well-integrated programs becomes evenly distributed. Because we not only want to know the predicted score of this method, but are also interested in how the score

of this method varies over different training and testing sets, we split our training data into 5 folds. From this estimation we find that over the 5 folds our Logistic Regression model predicts a program's integration level with an average accuracy level of 0.83, which does not vary at all over the five folds. Although this high accuracy and low variation is promising in terms of the predictive power and adaptation of our model, we explore further whether this high level of accuracy is due to well-integrated programs being over-predicted. For example, if our classifier predicted that all programs were well-integrated than it would have an accuracy of 0.25. In order to diagnose this we consider the sensitivity and specificity of the model.

4.2.1 Diagnostics:

The sensitivity of a model considers the probability that a positive observation was correctly identified. In our scenario, this refers to the probability that a program was correctly identified as low-integrated. Similarly, the specificity of a model considers the probability that a negative observation was correctly identified. For our purposes, this refers to the probability a program was well-identified as having its alumni well-integrated into the job market. We estimate the sensitivity and specificity of a model through the share of low-integrated programs and high-integrated programs that were correctly identified. Because our data set is highly unbalanced, if we don't adjust class weights we expect the specificity of our model to be much higher than the sensitivity.

The first column of Table 1 specifies the sensitivity and specificity of our Logistic Regression model when we don't adjust class weights. As expected, the prediction of well-integrated programs is much more accurate than that of low-integrated programs. In fact, using a null-prediction model which predicted whether program's were well-integrated with probability 0.5 would work just as well in predicting low-integrated programs as our Logistic Regression Model.

Table 1: Comparison of Logistic Regression Models by Performance Metric

	Unbalanced Model	**Balanced Model**	**Adjusted Threshold (0.3)**
Accuracy	0.83	0.78	0.79
Sensitivity	0.50	0.71	0.66
Specificity	0.93	0.73	0.79

In order to increase the sensitivity of our model we consider a Logistic Regression model where classes are artificially weighted such that low-integrated and well-integrated programs are equally considered in the minimization of (3). Similarly to as done for the simple logistic regression model, we split our data into five folds such that we can consider how the accuracy of our model varies over different training data. As presented in column 2 of Table 1, the average accuracy of the model over the five folds decreases to 0.78, whereas the variance in accuracy over the five folds is of 0.0001. The reason for this decreased accuracy is that the increased sensitivity of the model, seen to increase to 0.71 in Table 1 also decreases the specificity of the model, which decreases to 0.79. Although the increase in the model's sensitivity is higher than the decrease in the model's specificity, the overall accuracy of the model is still predicted to decrease as well-integrated programs represent a higher share of observations. This illustrates a trade-off in increasing the model's sensitivity. Key to whether such an adjustment should be made is how important it is for us to identify low-integrated programs as opposed to well-integrated programs. We can argue for both sides. For example, if a student was looking at this study whilst considering which Master's to pursue, it is likely that the cost of incorrectly identifying a low-integrated program to be well-integrated is higher than the opposite. On the other hand, if a school was using this study to determine whether programs should be shut down due to low-integration into the labor market, one could argue that a wrong classification of a well-integrated program as low-integrated program is more costly than the opposite.

Overall, as a way to balance the specificity, sensitivity, and accuracy of a model, we can control for the threshold under which a program is considered low-integrated; though this has to be done somewhat manually in python. Automatically, under the `LogisticRegression Pipeline` in python, an observation is predicted to belong to class $Y = 1$, if $P(Y = 1|X) > 0.5$, and to $Y = 0$ otherwise. However, because in an unbalanced model where our data does not perfectly fit a logistic distribution probabilities are skewed such that the majority class is favoured, if we lower the threshold to which we assign an observation to class $Y = 1$ (assuming that $Y = 1$ is the minority class), then we achieve something similar to what was done with the artificial class weightings. Yet, when controlling for the probability threshold we are able to balance between sensitivity and specificity to a greater degree.

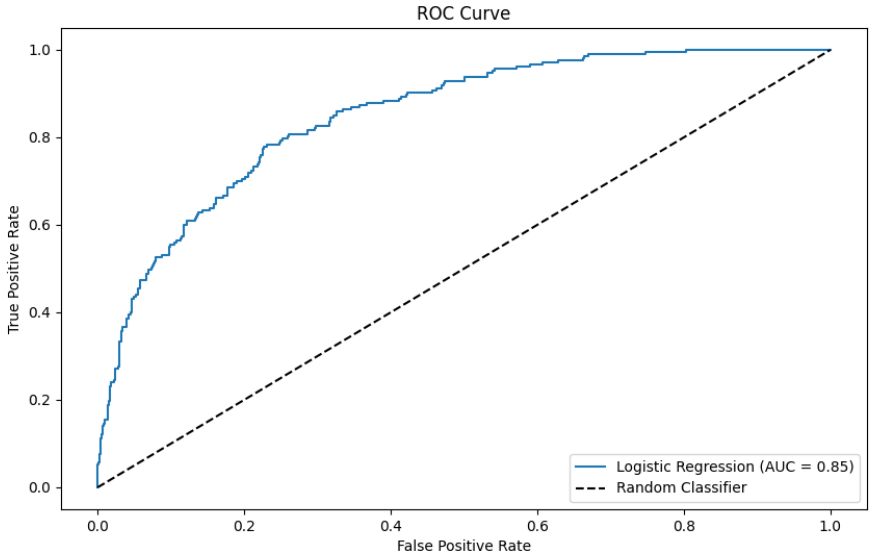

Figure 10: ROC Curve of Logistic Regression

In our data set we run a cross validation analysis of our Logistic Regression model under six different thresholds for predicting $Y = 1$, namely 0.2, 0.25, 0.3, 0.35, 0.4, and 0.45 over 5 folds of our training data. Note that we do this for a Logistic Regression model where classes have not been re-weighted. In Table 1 we present the accuracy, specificity, and sensitivity for a threshold of 0.3, which we deemed to be the optimal threshold between the six folds in our cross validation analysis. Between the six thresholds tested, we deemed this threshold to be optimal as it increased both the sensitivity and specificity of the balanced model in cross validation, though the same was not true for the testing data set used to produce Table 1.

In this same spirit, we also present Figure 10 which presents the ROC curve for our unbalanced Logistic Regression model under different thresholds for setting $Y = 1$. Traditionally a ROC curve, in addition to the calculated value under the curve (AUC), is used to evaluate the overall goodness of fit of a model. ROC curves which hug the top, left corners of the plot usually represent better models as they represent models with the possibility of having high sensitivity (top) and high specificity (left). Given that Figures 10 and 11 don't suggest our two classes are extremely well-separated, an AUC of 0.85 seems satisfactory.

4.3 Random Forest

Alternatively, we try to predict the labor market insertion rate for a given combination of discipline, university and year as a binary outcome using a non-linear model.

A random forest model learns patterns that learn variations in insertion rates across the different combinations. Essentially, it builds many decision trees and each tree splits the data based on features like discipline or the percentage of students on scholarships. The final prediction is the average across trees. We chose this model because it is more robust to overfitting compared to a single deep decision tree, and is naturally capable of capturing non-linear interactions between features. Machine learning models like random forest cannot handle non-numerical categorical variables. These are converted into dummy variables using one-hot encoding.

Regarding model specification, we assigned class weights to be balanced. This instructs the algorithm to internally upweight the minority class (vulnerable programmes, class 1) during training. This is important because, with only 25.4% of observations in class 1, an uncorrected model could achieve high accuracy simply by predicting class 0 almost all the time. Additionally, to ensure that each combination is represented relative to the size of its discipline, we use sampling weights. It tells the model to treat certain observations as more important than others during training. A program belonging to a large, well-represented discipline should carry more influence over the learned decision boundaries than a combination from a very small or niche discipline.

The data is split 80% for training and 20% for testing, yielding 6,568 training samples and 1,642 test samples.

Crucially, stratification is applied, meaning the split preserves the 25.4% / 74.6% class ratio in both the training and test sets.

Rather than using default settings, we perform hyperparameter tuning via a Grid Search with cross-validation to find the combination of hyperparameters that produces the best model. The search evaluates 32 combinations for the number of trees in the forest and the maximum depth among others. Each combination is evaluated using 5-fold cross-validation: the training set is split into 5 equal folds, the model is trained on 4 and validated on the 5th, cycling through all combinations. This gives a reliable estimate of out-of-sample performance for each hyperparameter set without touching the test set. The scoring metric is F1 score, which is the harmonic mean of precision and recall. For imbalanced classification, F1 is preferable to accuracy because it penalises both missing vulnerable programmes (low recall) and falsely flagging safe ones (low precision). As seen in Table 7 in the Appendix, the best configuration found has a cross-validated F1 score of 0.661. Hence, the model performs reasonably well, but is not highly accurate.

4.3.1 Evaluating performance

Table 2 presents the classification report on the test set to evaluate the prediction performance. Firstly, the accuracy informs us that the model correctly classifies 79% of programs overall. This gives an idea of the overall performance. We would like to focus on measuring how often a predicted vulnerable program is truly vulnerable, by looking at the precision. When the model predicts a program is vulnerable, it is correct about 57% of the time. This suggests that there are many false positives (programs flagged as vulnerable that are actually not). Moreover, the recall measures how many of the truly vulnerable programs are caught. It successfully identifies about 70%, but still misses about 30%. Lastly, the ROC-AUC of 0.86 is a strong result, indicating the model has good overall discriminative power across all possible classification thresholds.

Regarding performance on high insertion rate, the model performs very well on identifying programs that are not vulnerable. Therefore, random forest is better at confirming safe programs than detecting vulnerable ones. This result is confirmed when looking at the confusion matrix in Table 3.

Table 2: Classification Report on Test Set

Class	**Precision**	**Recall**	**F1-score**
0	0.89	0.82	0.85
1	0.57	0.70	0.63
Accuracy		0.79	
ROC-AUC		0.86	

Table 3: Confusion Matrix on Test Set

	Predicted 0	**Predicted 1**
Actual 0	1004	220
Actual 1	125	293

4.3.2 Feature interpretation using feature importance & SHAP analysis

We use two complementary approaches to feature interpretability. First, feature importance from the Random Forest itself rank each variable by how much it reduces impurity across all splits and all trees. It answers the question: which features does the model rely on most, across all predictions? Thus, it indicates each feature's overall contribution to the model's decision-making. The top predictor is employment situation 30 months post-graduation. The model is effectively learning that the trajectory of a cohort of graduates over time is the strongest signal of whether a program produces vulnerable outcomes. Programs where graduates are still in unstable employment situations 30 months out are much more likely to be classified as at-risk. The second most important feature indicates whether the program belongs to the teacher training (Masters enseignement) department. The teacher training programs in France are closely tied to competitive civil service examinations (Agrégation), meaning professional integration depends heavily on passing a national exam rather than simply holding a master's degree. The proportion of women enrolled in a program is the most important continuous demographic variable in the model. This likely reflects occupational segregation. In fact, programs with a high share of women tend to be concentrated in lower-paid or harder to enter fields.

The key limitation of feature importance is that importance scores are not directional. Indeed, they solely suggest whether a feature matters for prediction, but not whether high values increase or decrease vulnerability. That is where SHAP comes in. A positive SHAP value means that feature pushed the prediction toward vulnerability, while a negative value means it pushed away from it. The summary plot in Figure 11 shows both the

Table 4: Top 10 Most Important Features

Feature	Importance
situation_30 mois après le diplôme	0.098
domaine_Masters enseignement	0.081
domaine_Sciences humaines et sociales	0.068
femmes	0.067
diplome_MASTER LMD	0.059
discipline_Masters enseignement	0.053
de_diplomes_boursiers	0.051
taux_de_chomage_regional	0.050
annee	0.048
emplois_exterieurs_a_la_region_de_luniversite	0.044

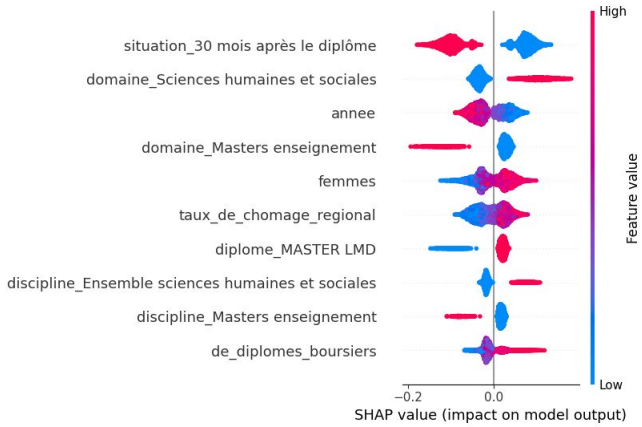

Figure 11: SHAP plot

magnitude and direction of each feature's effect across the training set, providing a much richer picture of what drives the model's decisions than feature importance alone.

Looking at the SHAP summary plot in Figure 11 which orders features by mean absolute SHAP value, the employment status after 30 months of graduation indicates a very high negative SHAP value. Being in teacher training consistently pushes toward vulnerability. A high female share in a program reinforces low integration prediction. Moreover, a high share of scholarship students and regional unemployment increases the likelihood of being classified as low integration.

4.4 Comparison between Logistic Regression and Random Forest

Table 5 summarises the performance of the two binary classifiers evaluated on the held-out test set.

Table 5: Performance Comparison: Logistic Regression vs. Random Forest

Metric	Logistic Regression	Random Forest
Accuracy	0.79	0.79
Sensitivity (Recall, class 1)	0.66	0.70
Specificity (Recall, class 0)	0.79	0.82
Precision (class 1)	–	0.57
F1-score (class 1)	–	0.63
ROC-AUC	0.85	0.86

Both models achieve comparable overall accuracy (0.79), yet they differ in meaningful ways. The Random Forest outperforms Logistic Regression on sensitivity (0.70 vs. 0.66), meaning it correctly identifies a larger share

of truly vulnerable programmes. To appreciate why this matters, consider the two types of misclassification errors the model can make. A false negative occurs when a vulnerable programme is incorrectly predicted as well-integrated: a student relying on this analysis would choose that master's programme under the false impression that its graduates integrate well into the labour market, only to face poor employment prospects upon graduation. A false positive, by contrast, occurs when a well-integrated programme is flagged as vulnerable: the student unnecessarily avoids a programme that would in fact have served them well. As discussed in Section 4.2, the relative cost of these two errors depends heavily on the end user. From a student's perspective, a false negative is arguably more damaging, as it leads to a poorly informed enrolment decision. From a university's perspective, a false positive may be equally costly, as it could unjustly trigger the review or closure of a healthy programme. Given this asymmetry, a model with higher sensitivity such as the Random Forest is preferable when the analysis is intended to protect prospective students, whereas Logistic Regression with an adjusted threshold may be more appropriate in an institutional policy context. The near-identical ROC-AUC scores (0.86 vs. 0.85) confirm that both models have similar overall discriminative power, and that the difference in sensitivity reflects a shift in the precision-recall trade-off rather than a fundamental difference in predictive capacity.

The two models also differ in their interpretability. Logistic Regression produces directly interpretable coefficient estimates: each predictor's effect on the log-odds of vulnerability can be read off and communicated clearly to policymakers. Random Forest, by contrast, is a black-box ensemble method that requires post-hoc tools such as the SHAP analysis presented in Figure 11 to recover interpretability. Importantly, both approaches converge on similar conclusions: being surveyed 30 months after graduation, belonging to the Humanities & Social Sciences or teacher training domains, a high share of women, and elevated regional unemployment all consistently emerge as the strongest drivers of vulnerability. This agreement across two methodologically distinct models, one parametric and linear, the other a non-linear ensemble, strengthens the credibility of these findings and suggests that the identified risk factors are robust rather than an artefact of any particular modelling choice.

A further distinction lies in how each model handles non-linearities. Logistic Regression assumes a linear relationship between the log-odds of vulnerability and the predictors, a restriction that may be too strong given the complex interactions identified in Section 2. Random Forest relaxes this assumption entirely, learning flexible decision boundaries through recursive partitioning, which likely explains the modest but consistent improvement in recall.

Finally, the Random Forest requires substantially more computational resources and hyperparameter tuning (grid search over 32 combinations with 5-fold cross-validation), whereas Logistic Regression is estimated rapidly and its threshold can be adjusted manually to balance sensitivity and specificity. Given the similarity in final performance, Logistic Regression remains a strong baseline that is easier to deploy and communicate, while Random Forest is preferable when maximizing the detection of vulnerable programs is the primary objective.

5 Conclusion

This report predicted professional integration vulnerability in French master's programs using data from the French Ministry of Higher Education covering 2010–2020. The analysis proceeded in two stages: predicting the insertion rate as a continuous variable, then reframing the problem as binary classification identifying programs falling below the 25th percentile threshold of 86%.

Linear, Ridge, and Lasso regression models all performed similarly, with penalization providing negligible improvement. This confirmed that linear models are insufficient to capture the complexity of insertion rate determinants, and motivated the shift to binary classification.

Both binary classifiers achieved comparable accuracy and ROC-AUC. The Random Forest modestly outperformed Logistic Regression on recall, catching more truly vulnerable programs. Logistic Regression remains preferable where interpretability and ease of communication matter most.

Finally, we found the main drivers of low insertion rate to be employment situation 30 months post-graduation, department of study, particularly teacher training and humanities, high female share, type of Master LMD, and regional unemployment. These reflect structural inequalities in the French labor market rather than program-level failures. Effective policy intervention therefore requires addressing these structural conditions rather than simply flagging individual programs.

Variable Name	Description
annee	Year
diplome	Diploma (type of masters)
numero_de_l_etablissement	Identification number of university
etablissement	University name
etablissementactuel	Current university name
code_de_l_academie	Academy code — corresponds to the location of the university
academie	Location of university (by city)
code_du_domaine	Code for department of study
domaine	Department of study name
discipline	Specific field of study within department
situation	How many months after masters completion were the individuals surveyed
nombre_de_response	Number of responses for discipline + university + year combination (unit of observation)
taux_de_response	Response rate for survey
poids_de_la_discipline	Sampling weight which should be given to the discipline + university + year combination
taux_dinsertion	Percentage of applicants who have integrated themselves into the labor market (note: a lot of 'ns' because the ministry deemed there to not be enough responses to accurately estimate this)
taux_d_emploi	Percentage of which are employed within France (a lot of ns/missing data)
taux_d_emploi_salarie - _en_france	Percentage who are working a paid job in France
emplois_cadre_ou - professions_intermediaires	Percentage who are employed in an 'upper class' or 'intermediary' job
emplois_stables	Share of employees who are in a stable job
emplois_a_temps_plein	Share of employees who are in full-time work position
salaire_net_median - des_emplois_a_temps_plein	Median monthly salary of those who are working
salaire_brut_annuel_estime	Median yearly salary
de_diplomes_boursiers	Percentage of students who are on a scholarship (could argue this is a proxy for school connections/finances/price)
taux_de_chomage_regionel	Unemployment rate of region
salaire_net_mensuel - _median_regional	Median monthly salary of young people in the region
emploi_cadre	Percentage who are in an 'upper class' job
emplois_exterieurs_exterieurs - _a_la_region_de_luniversite	Percentage of people working a job located outside the university's region
femme	Share of women in discipline + university + year combination
salaire_net_mensuel_regional - _1er_quartile	First quantile of monthly salary of young people in the region
salaire_net_mensuel_regional - _3eme_quartile	Third quantile of monthly salary of young people in the region
cle_disc	Discipline and number of months after graduation identification code
cle_etab	Location and number of months after graduation identification code
id_paysage	Establishment and number of months after graduation identification code

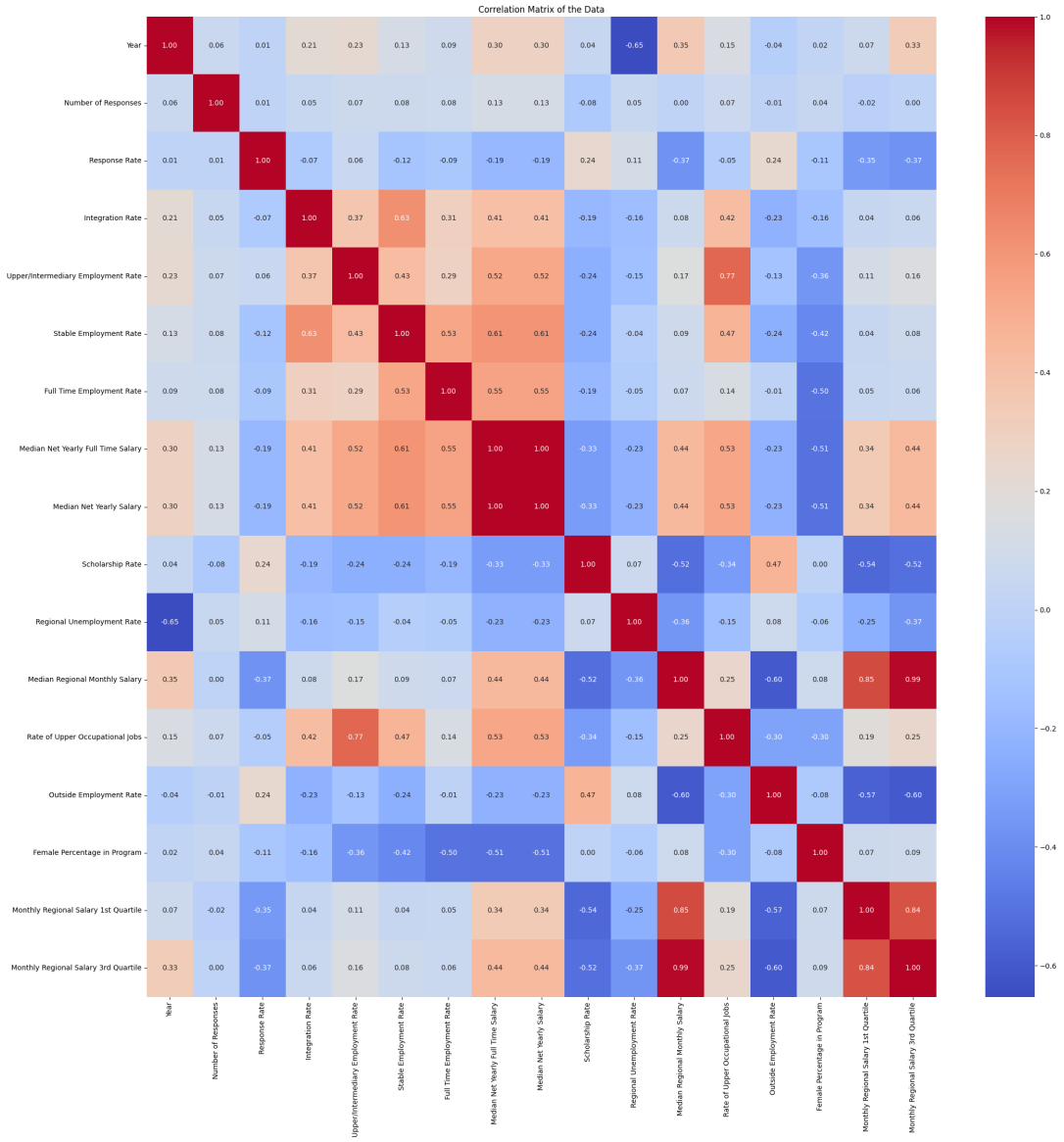

Figure 12: Correlation matrix over numerical features of our data set.

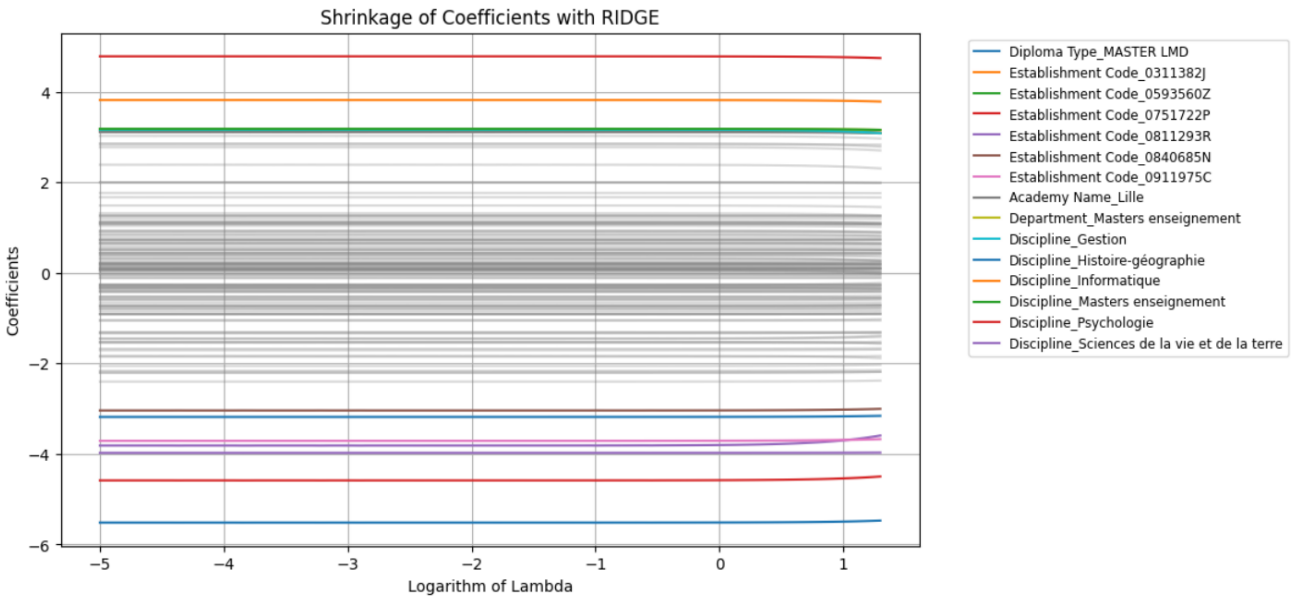

Figure 13: Estimates of parameters under ridge regressor as lambda increases ($\lambda \in [10^{-10}, 300]$). Estimates which stay above 3 in absolute value for all values of lambda considered are highlighted. Note that the logarithm considered is that of base 100.

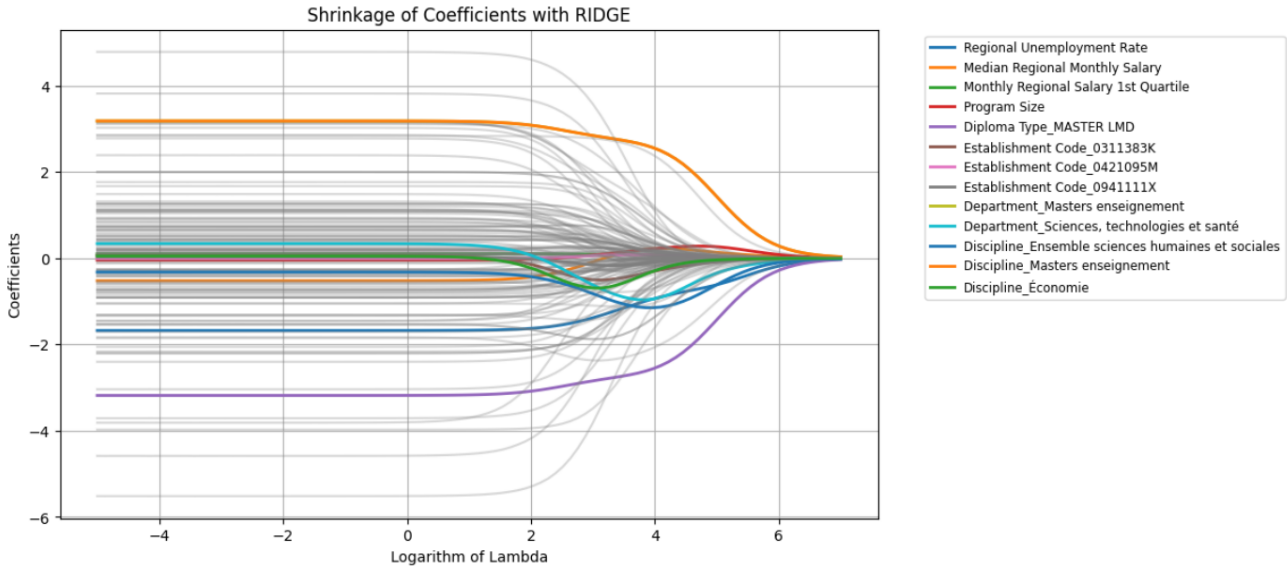

Figure 14: Estimates of parameters under ridge regressor as lambda increases ($\lambda \in [10^{-10}, 10^{14}]$). Estimates which correspond to those whose size was the greatest relative to their original estimate under the lowest λ value are highlighted. Note that the logarithm considered is that of base 100.

Table 7: Random Forest Hyperparameters and Performance

Parameter	**Value**
max_depth	20
max_features	sqrt
min_samples_leaf	4
min_samples_split	10
n_estimators	100
Cross-validated F1 score	0.661